PORTLINE IDP
Deployment Guide

A practical guide to local setup, Docker deployment, Linux VPS hosting, configuration, model caching and production operations.

SYSTEM COMPONENT	ROLE
FastAPI	Backend API and application services
PostgreSQL + AsyncPG	Persistent relational database
Redis	Rate limiting and token/cache support
Gemma 2 (2B)	Local language-model inference
Vite frontend	Web dashboard and user interface
Docker / Linux services	Application deployment and process management

Document basis: Consolidated from the two deployment documents supplied in this conversation. Commands, service names and configuration examples should be checked against the current repository before production use.

1. Before You Begin

This guide covers two deployment paths: running the application locally with Docker Compose, and hosting it on a Linux server. Choose the path that matches your environment. The Docker path is generally the simplest starting point.

1.1 Hardware requirements

Resource	Minimum (development/testing)	Recommended (production)
CPU	4-core x86-64 processor	8 or more cores; strong single-core performance
RAM	8 GB	16–32 GB
Storage	30 GB SSD	100 GB or more NVMe SSD
Hugging Face model cache	About 3 GB free	About 10 GB free for model snapshots

Gemma 2 (2B) model weights are stored in the Hugging Face cache. Allow extra disk space if you keep multiple model revisions or other cached models.

1.2 Software and access you need

- Docker Desktop with Docker Compose for local container deployment.
- Git to clone the repository.
- Node.js and npm to install and run the frontend.
- A Hugging Face access token (HF_TOKEN) if required to download the Gemma model.
- For Linux VPS deployment: Ubuntu 22.04 LTS, RHEL or Debian, plus Python, a virtual environment, PostgreSQL, Redis, Nginx and systemd.

2. Local Deployment with Docker Compose

Follow these steps in order. Run commands from the appropriate project directory. The examples use the repository and service names shown in the supplied deployment material.

Step 1 — Clone the repository

```
git clone https://github.com/portline-customs/customs-idp-engine.git
cd customs-idp-engine
```

Step 2 — Create the environment file

```
cp .env.example .env
```

On Windows PowerShell, use `Copy-Item .env.example .env` if the `cp` command is unavailable. Open `.env` and set the required values, especially the secrets and Hugging Face token. Do not commit this file to Git.

Step 3 — Build and start the containers

```
docker compose up -d --build
```

This starts the services defined in your Compose file. The supplied configuration includes PostgreSQL, Redis and the API. It also mounts persistent storage and a Hugging Face cache directory.

Step 4 — Download the Gemma 2 (2B) model

Pre-downloading the model avoids making the API wait for a large model download during normal application startup. The following command runs the download inside the API container and uses HF_TOKEN from its environment:

```
docker exec -it customs_idp_backend-api-1 python -c "import os; from huggingface_hub import
snapshot_download; token = os.getenv('HF_TOKEN'); print('--> Downloading Gemma 2 (2B)...');
snapshot_download(repo_id='google/gemma-2-2b-it', token=token, max_workers=4); print('--> Download
Complete!')"
```

```
docker compose restart api
```

The container name in the command must match the actual API container name on your machine. Check it with `docker compose ps` if the command reports that the container cannot be found.

Step 5 — Verify the model cache

```
docker exec -it customs_idp_backend-api-1 python -c "from huggingface_hub import scan_cache_dir; [print(
f'Model: {r.repo_id} | Size: {r.size_on_disk_str}') for r in scan_cache_dir().repos]"
```

Step 6 — Start the frontend

```
cd portline-frontend
npm install
npm run dev
```

Step 7 — Open the application

Service	Local address
Frontend dashboard	http://localhost:5173
Backend API documentation (Swagger)	http://localhost:8000/docs
Backend health check	http://localhost:8000/health

3. How the Docker Setup Works

Service	Purpose	Persistent data / notes
db (PostgreSQL 16)	Stores application data.	The pgdata volume keeps database files across container restarts.
redis (Redis 7)	Supports rate limiting and token cache.	Configured with a 64 MB memory limit in the supplied example.
api (FastAPI)	Runs the backend and application logic.	Uploads are stored in the uploads volume; model files use the mounted Hugging Face cache.

3.1 Important Compose configuration

The supplied Compose example configures the API with database and Redis connection strings, an Ollama base URL, and HF_TOKEN. It mounts the host Hugging Face cache into `/root/.cache/huggingface` inside the container.

```
services:
  db:
    image: postgres:16-alpine
    environment:
      POSTGRES_USER: customs
      POSTGRES_PASSWORD: customs
      POSTGRES_DB: customs_idp
    ports:
      - "5432:5432"
    volumes:
      - pgdata:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U customs -d customs_idp"]
      interval: 5s
      retries: 10

  redis:
    image: redis:7-alpine
    command: ["redis-server", "--save", "", "--maxmemory", "64mb",
              "--maxmemory-policy", "noeviction"]
    healthcheck:
      test: ["CMD", "redis-cli", "ping"]
      interval: 5s
      retries: 10

  api:
    build: .
    env_file:
      - .env
    environment:
      DATABASE_URL: postgresql+asyncpg://customs:customs@db:5432/customs_idp
      REDIS_URL: redis://redis:6379/0
      OLLAMA_BASE_URL: http://ollama:11434
      HF_TOKEN: ${HF_TOKEN}
    ports:
      - "8000:8000"
    volumes:
      - uploads:/app/storage
      - ${USERPROFILE}/.cache/huggingface:/root/.cache/huggingface
    depends_on:
      db:
        condition: service_healthy
      redis:
        condition: service_healthy

volumes:
  pgdata:
  uploads:
```

Configuration checks before using this example

• The Compose snippet refers to `http://ollama:11434`, but does not define an `ollama` service. Confirm that another service provides this hostname, or remove/change the setting if the application does not use it.

• The sample database username and password are demonstration values. Replace them with strong, unique credentials and update DATABASE_URL to match.

• The Hugging Face cache mount uses the Windows `USERPROFILE` variable. On Linux, use the correct host cache path for that server.

• For production, avoid exposing PostgreSQL or Redis ports publicly. Bind them to localhost or keep them on a private container network unless external access is explicitly required.

3.2 Non-blocking model initialization

Loading a language model into CPU memory can take time. If it happens directly during FastAPI startup, the server may not respond promptly while the model loads. The supplied application uses a deferred background task so the API can start accepting requests while model initialization runs separately.

```
@asynccontextmanager
async def lifespan(_: FastAPI):
    init_store()

    if settings.AUTO_CREATE_TABLES:
        await create_all_tables()
    async with async_session_factory() as session:
        await ensure_super_admin(session)

    # Start model initialization in a background thread.
    loop = asyncio.get_running_loop()
    loop.call_soon(
        lambda: asyncio.create_task(asyncio.to_thread(init_gemma_model))
    )

    yield
    await close_store()
    await engine.dispose()
```

Keep this logic in the FastAPI lifespan handler in `app/main.py` if it matches the current application implementation. Confirm that the model-loading function and imports exist in your codebase.

4. Production Deployment on a Linux VPS

For a Linux server, the supplied guide uses systemd to keep the FastAPI process running and Nginx as a reverse proxy. Nginx receives HTTPS traffic and forwards requests to the API running locally on port 8000.

4.1 Create the systemd service

Create the service file at `/etc/systemd/system/portline-api.service`:

```
[Unit]
Description=Portline IDP FastAPI Gateway Engine
After=network.target postgresql.service redis.service

[Service]
User=www-data
Group=www-data
WorkingDirectory=/var/www/customs_idp_backend
EnvironmentFile=/var/www/customs_idp_backend/.env
ExecStart=/var/www/customs_idp_backend/venv/bin/uvicorn app.main:app --host 0.0.0.0 --port 8000 --
workers 4
Restart=always
RestartSec=5

[Install]
WantedBy=multi-user.target
```

Then reload systemd and enable the service:

```
sudo systemctl daemon-reload
sudo systemctl enable portline-api
sudo systemctl start portline-api
sudo systemctl status portline-api
```

The paths, Linux user, virtual-environment location and worker count are examples from the supplied guide. Adjust them to the actual server layout and resource capacity.

4.2 Configure Nginx as a reverse proxy

Create an Nginx site configuration such as `/etc/nginx/sites-available/portline`. Replace the example hostname and certificate paths with values valid for your deployment.

```
server {
    listen 80;
    server_name api.portline-customs.internal;
    return 301 https://$host$request_uri;
}

server {
    listen 443 ssl http2;
    server_name api.portline-customs.internal;

    ssl_certificate /etc/letsencrypt/live/api.portline-customs.internal/fullchain.pem;
    ssl_certificate_key /etc/letsencrypt/live/api.portline-customs.internal/privkey.pem;

    client_max_body_size 50M;

    location / {
        proxy_pass http://127.0.0.1:8000;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }
}
```

Enable and test the Nginx configuration:

```
sudo ln -s /etc/nginx/sites-available/portline /etc/nginx/sites-enabled/portline
sudo nginx -t
sudo systemctl reload nginx
```

The certificate files must already exist and be valid. The example hostname uses an internal-style domain; use a publicly resolvable domain and an appropriate certificate if the service must be reachable over the public internet.

5. Deployment on Container Platforms

The supplied material also provides generic Docker commands for platforms such as AWS ECS, Kubernetes and Render. The exact deployment workflow differs by platform, but the image and runtime configuration follow the same general pattern.

5.1 Build the Docker image

```
docker build -t portline-idp-api:v1.2.0 .
```

5.2 Run the container

```
docker run -d -p 8000:8000 --env-file .env \
  -v /var/cache/huggingface:/root/.cache/huggingface \
  portline-idp-api:v1.2.0
```

5.3 Apply database migrations and start the API

```
alembic upgrade head && uvicorn app.main:app --host 0.0.0.0 --port $PORT
```

Set the platform's environment variables and persistent storage according to its requirements. Do not assume that a local filesystem path or volume mount is persistent on every hosting platform.

6. Environment Variables

Set these values in the server's environment or in a protected `.env` file. The table describes the variables listed in the supplied comprehensive guide.

Variable	Example / expected format	Purpose
ENVIRONMENT	production	Selects execution mode, such as development or production.
DATABASE_URL	postgresql+asyncpg://...	Async PostgreSQL connection string.
REDIS_URL	redis://redis:6379/0	Redis connection string for rate limiting and token cache.
SECRET_KEY	32+ character random secret	Signs JWT tokens.
AES_ENCRYPTION_KEY	64 hexadecimal characters	Encrypts sensitive PII and document metadata at rest.
HF_TOKEN	Hugging Face token	Authenticates access to Gemma 2 (2B) model weights when required.
SUPERADMIN_EMAIL	admin@example.com	Email address for the initial system administrator.
SUPERADMIN_INIT_PASSWORD	Strong unique password	Initial administrator bootstrap password.

Secret-handling rules

- Never commit `.env`, API tokens, encryption keys or passwords to source control.
- Use a secret manager or protected environment variables in production.
- Generate a unique, high-entropy SECRET_KEY and AES_ENCRYPTION_KEY. Do not reuse sample values.
- Restrict access to the environment file and ensure backups containing secrets are protected.
- If a token or password has been exposed, rotate it and update the deployment.

7. First-Time Accounts and Health Checks

The supplied guide lists two reviewer/demo roles. The original document includes example login credentials. Treat those credentials as public demonstration values: do not use them in a live deployment. Set new passwords before exposing the application, and disable or remove demo accounts that are not required.

Role	Example email	Access described in source
Super Administrator	admin@example.com	Full workspace, system audit and user controls.
Documentation Executive	demo@example.com	Document ingestion, gatekeeper review and AI Copilot.

Health endpoint

```
GET /health
```

The supplied guide states that this endpoint returns database status, Redis ping status and the active AI engine configuration. After deployment, call this endpoint and confirm that all required dependencies report healthy.

8. Production Readiness Checklist

- Use strong, unique database credentials and application secrets.

- Keep HF_TOKEN and all other credentials outside Git.
- Configure the frontend to use the intended API base URL, such as `import.meta.env.VITE_API_BASE_URL`, rather than relying on user-editable client-side values.
- Configure CORS in `app/main.py` to allow only authorized production frontend origins.
- Put Nginx or an equivalent load balancer/reverse proxy in front of the API and enforce HTTPS with TLS 1.2 or later.
- Redirect HTTP traffic to HTTPS.
- Keep database and Redis services on a private network; expose only the ports that must be public.
- Use persistent storage for PostgreSQL data, uploaded files and model cache where required.
- Verify the model is downloaded and available before relying on AI inference.
- Check logs, service status and `/health` after every deployment or restart.
- Back up the database and uploaded files, and test that restoration works.

9. Useful Operations Commands

Task	Command
List Compose services	docker compose ps
View all Compose logs	docker compose logs
Follow API logs	docker compose logs -f api
Restart API service	docker compose restart api
Stop Compose services	docker compose down
Check systemd service	sudo systemctl status portline-api
View systemd logs	sudo journalctl -u portline-api -f
Test Nginx config	sudo nginx -t

Final note

This document reorganizes and explains the two supplied deployment guides. It does not verify that the example repository, container names, paths, environment variables or platform settings still match the latest code. Before production deployment, compare every example with the current Docker Compose file, application settings, frontend configuration and hosting-provider requirements.